AOSP 进阶丛书 · 第四册 · ANDROID 13 (API 33)

# AMS 四大组件调度源码精读

Activity 冷启动十步、ClientTransaction 生命周期事务化、Service/Broadcast/Provider 同构骨架与欠条-兑现模型、computeOomAdj 优先级记账机器与 ANR 体系——逐段注释读懂调度中枢。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

# 目 录


前言：从"会用"到"会管"——调度中枢的解剖课 . . . . . 2

第一章 Activity 启动：十步冷启动全链路 . . . . . 2

第二章 ClientTransaction：生命周期事务化 . . . . . 6

第三章 Service：最安静的组件 . . . . . 8

第四章 Broadcast：系统的脉搏 . . . . . 10

第五章 ContentProvider：数据的跨进程门户 . . . . . 11

第六章 进程优先级：computeOomAdj 记账机器 . . . . . 12

第七章 深水区：ART 内存执行系统与 lmkd . . . . . 14

第八章 线程发动机：Handler/Looper/MessageQueue 全解 . . . . . 15

第九章 设计决策八问 . . . . . 16

第十章 实战：读一次真实启动的 trace . . . . . 17

附录 A 观测实操 . . . . . 18

附录 B 自检题 . . . . . 18

附录 C 崩溃/卡顿与调度的对照表 . . . . . 18

附录 D 快问快答（面试扫雷 20 条） . . . . . 19


**本册速览**

Activity 冷启动十步（含 realStart/performLaunch 逐段注释）
ClientTransaction 事务化：callbacks+终态，cycleToPath 补路径
四组件同构骨架：欠条-兑现 · ANR 五源 · computeOomAdj 记账机器
第七章 深水区（ART CC/MTERP/lmkd/冻结） · 第八章 线程发动机（Handler→epoll）
第九章 设计决策+三补全（PendingIntent/ActivityResult/ReceiverDispatcher）

# 前言：从"会用"到"会管"——调度中枢的解剖课

前三册走完了开机与通信两条主线，本册进入系统开发者的日常主战场：**AMS（ActivityManagerService）及其分管的四大组件调度**。你将看到 startActivity 背后的每一次跨进程往返、Service/Broadcast/Provider 三兄弟的同构骨架、进程优先级如何被一台"记账机器"实时核算，以及 ANR 从诞生到弹窗的全过程。

本册的代码讲解采用**逐段注释**体例：每段源码摘录都带中文行内注释（标注为"讲解注释"）+ 代码后的"逐段精读"编号解读，初学者可以逐行跟读。源码基线不变（本树 Android 13；注意本树 `ActivityStackSupervisor` 更名为 `ActivityTaskSupervisor`，写作时两处均有注明）。

# 第一章 Activity 启动：十步冷启动全链路

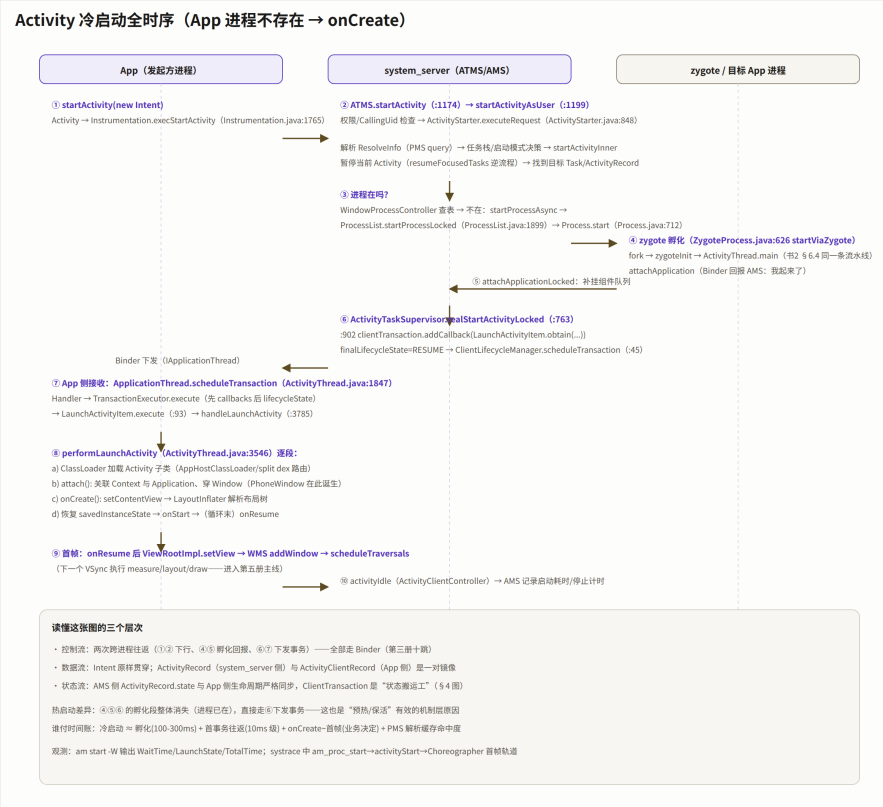



图 1 Activity 冷启动全时序：发起方 App、system_server、zygote/目标进程三泳道，十步逐跳

## 1.1 全链路鸟瞰

冷启动十步（图①~⑩）：`startActivity` → `Instrumentation.execStartActivity`（`Instrumentation.java:1765`）→ `ATMS.startActivity`（`ActivityTaskManagerService.java:1174`）→ `ActivityStarter.executeRequest`（`ActivityStarter.java:848`）→ 进程不存在则孵化（`ProcessList.startProcessLocked:1899` → zygote，书2 同款流水线）→ `attachApplication` 回报 →

ActivityTaskSupervisor.realStartActivityLocked（:763）组装事务 → Binder 下发 → App 侧 LaunchActivityItem.execute（:93）→ handleLaunchActivity（ActivityThread.java:3785）→ performLaunchActivity（:3546）。

## 1.2 executeRequest：守门与决策（逐段注释）

java frameworks/.../wm/ActivityStarter.java:848 起（节选+讲解注释）

```
private int executeRequest(Request request) {
// ── 第 1 段：身份与快速失败 ──────────────────────
// 讲解注释：所有跨进来的请求先"验明正身"。callingUid 来自 Binder（书3 §5.2），
// startAnyPerm 拥有者可以绕过组件导出检查（系统特权应用如 shell）。
...
// ── 第 2 段：解析目标 ──────────────────────────
// 讲解注释：queryIntentActivities 走 PMS（第六册）的组件倒排索引，
// 拿到 ActivityInfo；没有匹配直接返回 START_INTENT_NOT_RESOLVED。
...
// ── 第 3 段：权限闸门 ──────────────────────────
// 讲解注释：startActivity 权限、跨用户检查、抽象 URI 授权（UriGrantsManager，
// SystemServer Bootstrap 批先于 AMS 启动的意义在此）。
...
// ── 第 4 段：交给内层 ──────────────────────────
// 讲解注释：走 startActivityInner 做任务栈决策（launchMode/flags/任务亲和性），
// 决定"复用哪个 Task、放在哪、是否清栈"。
}
```

**设计洞察 · 逐段精读：executeRequest 的三重身份**

第 1 段是**安检员**（身份可信，其余免谈）；第 2 段是**翻译官**（把语义化的 Intent 翻译成确定的组件与类名）；第 3、4 段才是**调度员**。这个"先验证、再解析、后决策"的三段式是所有系统服务入口方法的通用骨架——与你在第三册看到的 servicemanager.addService 五道关卡同构。

## 1.3 realStartActivityLocked：组装事务（真实源码逐段注释）

java frameworks/.../wm/ActivityTaskSupervisor.java:895-933（节选，注释为讲解注释）

```
// Create activity launch transaction.
// 讲解注释：ClientTransaction.obtain 从对象池取事务；proc.getThread() 就是
// 目标进程的 IApplicationThread Binder 代理——事务的"收件人"。
final ClientTransaction clientTransaction = ClientTransaction.obtain(
proc.getThread(), r.token);
final boolean isTransitionForward = r.isTransitionForward();
final IBinder fragmentToken = r.getTaskFragment().getFragmentToken();
// 讲解注释：LaunchActivityItem 装的是"开工全套材料"：Intent、ActivityInfo、
// 全局/覆盖 Configuration（屏幕旋转等）、savedState（进程被杀后的现场恢复）、
// 结果与新 Intent、ProfilerInfo……App 侧 onCreate 需要的一切都在包里。
clientTransaction.addCallback(LaunchActivityItem.obtain(new Intent(r.intent),
System.identityHashCode(r), r.info,
mergedConfiguration.getGlobalConfiguration(),
mergedConfiguration.getOverrideConfiguration(), r.compat,
r.getFilteredReferrer(r.launchedFromPackage), task.voiceInteractor,
proc.getReportedProcState(), r.getSavedState(), r.getPersistentSavedState(),
results, newIntents, r.takeOptions(), isTransitionForward,
```

```
proc.createProfilerInfoIfNeeded(), r.assistToken, activityClientController,
r.shareableActivityToken, r.getLaunchedFromBubble(), fragmentToken));
// Set desired final state.
// 讲解注释：终态二选一——正常路径 ResumeActivityItem；先不显示（如启动到后台）
// 则 PauseActivityItem。App 侧会沿状态机补齐中间态（onCreate→onStart→onResume）。
final ActivityLifecycleItem lifecycleItem;
if (andResume) {
lifecycleItem = ResumeActivityItem.obtain(isTransitionForward);
} else {
lifecycleItem = PauseActivityItem.obtain();
}
clientTransaction.setLifecycleStateRequest(lifecycleItem);
// Schedule transaction.
// 讲解注释：ClientLifecycleManager.scheduleTransaction（:45）→ Binder 下发，
// 书3 十跳的又一次具体旅行。
mService.getLifecycleManager().scheduleTransaction(clientTransaction);
```

逐段精读三要点：①**事务的收件人是进程不是 Activity**——token（`r.token`）才是 Activity 的镜像句柄，同一进程的多个 Activity 共享一条 Binder 通道；②**"材料"与"终态"分离**——callbacks 描述"发生什么"，lifecycleState 描述"最终到哪"；③**andResume 分支**解释了一个常见现象：通知栏点进来的 Activity 可能走到 onCreate 但不 onResume（启动到后台）。

## 1.4 performLaunchActivity：App 侧的出生仪式（真实源码逐段注释）

java frameworks/base/core/java/android/app/ActivityThread.java:3546-3605（节选）

```
private Activity performLaunchActivity(ActivityClientRecord r, Intent customIntent) {
ActivityInfo aInfo = r.activityInfo;
// 讲解注释①：LoadedApk——本 APK 的"资源+类加载器"上下文；
// CONTEXT_INCLUDE_CODE：把 APK 的 dex 挂进 ClassLoader（split dex 路由于此）。
if (r.packageInfo == null) {
r.packageInfo = getPackageInfo(aInfo.applicationInfo, r.compatInfo,
Context.CONTEXT_INCLUDE_CODE);
}
ComponentName component = r.intent.getComponent();
...
// 讲解注释②：activity-alias 场景——manifest 的 targetActivity 指向真实类。
if (r.activityInfo.targetActivity != null) {
component = new ComponentName(...targetActivity);
}
ContextImpl appContext = createBaseContextForActivity(r);
Activity activity = null;
try {
// 讲解注释③：反射创建 Activity 实例。你的无参构造函数就是在这里被调用的；
// 此刻它还没有 Context——所以构造函数里不能调用任何需要 Context 的 API。
java.lang.ClassLoader cl = appContext.getClassLoader();
activity = mInstrumentation.newActivity(
cl, component.getClassName(), r.intent);
...
} catch (Exception e) { ... throw new RuntimeException("Unable to instantiate activity ...");
}
try {
// 讲解注释④：确保 Application 存在（首启动时在此 onCreate）；
// makeApplicationInner(false,...) 表示非重载路径。
```

```
Application app = r.packageInfo.makeApplicationInner(false, mInstrumentation);
...
if (activity != null) {
// 讲解注释⑤：attach——注入 Context/Window（PhoneWindow 诞生地）、
// 关联 Application 与 Intent；随后就是 onCreate（setContentView 的家）。
...
}
...
```

逐段精读：③与⑤合起来解释了经典面试题"Activity 构造函数与 onCreate 里各能做什么"——构造时只有裸对象，attach 后才有 Context 体系。④说明 Application.onCreate 的真实触发时机：**首个组件启动路径上，而不是进程启动时**（attach 时若未创建则补）。

**源码锚点 · 第一章锚点速查**

<font face="MonoFT" size="9">Instrumentation.java:1765</font> execStartActivity / <font face="MonoFT" size="9">ActivityTaskManagerService.java:1174/1199</font> startActivity(AsUser)<br/><font face="MonoFT" size="9">ActivityStarter.java:848</font> executeRequest / <font face="MonoFT" size="9">ActivityTaskSupervisor.java:763</font> realStartActivityLocked、<font face="MonoFT" size="9">:902</font> LaunchActivityItem 组装<br/><font face="MonoFT" size="9">ActivityThread.java:1847</font> scheduleTransaction / <font face="MonoFT" size="9">:3785</font> handleLaunchActivity / <font face="MonoFT" size="9">:3546</font> performLaunchActivity

## 1.5 任务栈决策：launchMode 与 flags 的裁决现场

`startActivityInner`（`ActivityStarter.java:1646` 注释所指）是 launchMode 的"法庭"。判案顺序固定，初学者背下这个顺序就不会迷路：

text 任务栈裁决顺序（简化自 ActivityStarter/Task 处理逻辑）

```
1 FLAG_ACTIVITY_NEW_TASK? ──是→ 找 taskAffinity 匹配的目标 Task（没有则新建）
2 目标 Task 定下后，在栈顶找"可复用"的实例：
• singleTop：栈顶同组件 → 走 onNewIntent（不新建）
• singleTask：栈内存在 → 清其上方全部实例 + onNewIntent
• singleInstance：独占一个 Task（此 Task 永远只有它）
3 FLAG_ACTIVITY_CLEAR_TOP：栈内存在 → 销毁其上实例（与 launchMode 组合出 8 种行为）
4 默认 standard：无条件 new 实例压栈
5 结果落账：Task 组织进 RootWindowContainer；ActivityRecord.setState(INITIALIZING)
```

**设计洞察 · 逐段精读：为什么决策这么复杂**

因为 Android 的"任务"是**用户视角的聚合**（最近任务卡片）而非进程视角——跨进程的 Activity 能进同一张卡片，靠 taskAffinity 声明归属。裁决顺序里 NEW_TASK 排第一，因为它决定"去哪个法庭"；launchMode 决定"能不能复用被告席"。排障口诀：**launchMode 看 Task 内，flags 看 Task 间**。

## 1.6 attachApplicationLocked：孵化后的"总兑现"时刻

新进程回报 `attachApplication` 后，AMS 在 `attachApplicationLocked` 里做一次**集中兑现**（本册最重要的"还款日"）：

java am/ActivityManagerService.java attachApplicationLocked（骨架+讲解注释）

```
private boolean attachApplicationLocked(...) {
// 讲解注释①：进程记账入库：pid→ProcessRecord 绑定、
// proc.setThread(thread) 记下 IApplicationThread 代理（此后才有"收件人"）。
// 讲解注释②：优先级初算：generateProcessForLockedUpActivities 等路径
// 视 pending 组件给初始 state（别让新进程一出生就被 lmkd 盯上）。
// 讲解注释③：兑现三大欠条：
// a) mPendingServices → realStartServiceLocked（第三章）
// b) mPendingBroadcast → processCurBroadcastLocked（第四章）
// c) mLaunchingProviders → publishContentProviders（第五章）
// 讲解注释④：再跑一次 OomAdjuster（状态变了，待遇重算）。
// 讲解注释⑤：stable/unstable provider 计数、backup agent 等收尾。
}
```

逐段精读：**为什么集中在这里？** 因为 IApplicationThread 就位是"一切下发的前提"——四组件的欠条都等这一个信号。把这个函数读熟，你同时复习了四条组件链的交汇点，这也是 AMS 面试的高频"枢纽题"。

# 第二章 ClientTransaction：生命周期事务化




图 2 ClientTransaction 模型：callbacks+终态、App 侧 TransactionExecutor 的补态执行与设计动机

## 2.1 TransactionExecutor 逐段注释（App 侧执行器）

java frameworks/base/core/java/android/app/servertransaction/TransactionExecutor.java（骨架+讲解注释）

```
public void execute(ClientTransaction transaction) {
```

```
// 讲解注释①：先跑 callbacks（LaunchActivityItem/NewIntentItem…按序），
// 每个 Item 自己知道该调 ActivityThread 的哪个 handleXxx。
executeCallbacks(transaction);
// 讲解注释②：再执行终态请求。注意 executeLifecycleState 内部会先
// cycleToPath——从当前状态沿状态机"走"到目标状态，中间态一个不跳。
executeLifecycleState(transaction);
...
}
private void executeLifecycleState(ClientTransaction transaction) {
...
// 讲解注释③：pathToTargetState 生成要补的事件序列。
// 例：当前 onCreate 完（CREATED），终态 RESUME → 补 onStart、onResume。
// 这就是"你永远不会收到没有 onStart 的 onResume"的机制保证。
cycleToPath(r, lifecycleItem.getTargetState(), true /* excludeLastState */, transaction);
// 讲解注释④：真正执行终态 Item（ResumeActivityItem→activityResumed），
// Item.postExecute 再回执 AMS（activityIdle 链在此闭环）。
lifecycleItem.execute(mTransactionHandler, token, mPendingActions);
...
}
```

逐段精读：③是整个事务模型的灵魂——**状态机路径由执行器推导**而非调用方拼装。AMS 只说"我要它到 RESUME"，怎么走是协议层保证的。这把 Android 9 之前"调度方拼事件序列"的责任反转成了"执行方补路径"，出错面骤减。

> **设计洞察 · 为什么 Android 9 要改造**
>
> 旧世界几十个 `scheduleXxx` 独立 Binder 调用，顺序与原子性全靠约定——配置变更场景常出现"先 resume 后 config"的闪烁。事务化后一次投递一批变更，App 侧 `cycleToPath` 保证状态机走连续路径（CREATE→RESUME 必经 START），系统侧可整批回放。**把"多次可重排的消息"变成"一次不可拆的事务"，是分布式系统对抗乱序的经典解法。**

App 侧执行器 `TransactionExecutor` 的规则值得背下：先逐项执行 callbacks（LaunchActivityItem、NewIntentItem…按序），再执行 lifecycleStateRequest（沿状态机补中间态），最后 postExecutionState 回执 AMS。`servertransaction/` 目录就是"生命周期指令集"——新增周期能力=新增一个 Item 类，协议不动。

## 2.2 屏幕旋转：一次"重建事务"的解剖

旋转不是"魔法重启"，而是三条事务的组合拳（`ConfigurationChangeItem` + `ActivityRelaunchItem`，均在 servertransaction/）：

- **能自处理**（manifest `configChanges=orientation`）→ 只发 `ActivityConfigurationChangeItem` → App 侧 `onConfigurationChanged`，View 树自己适配。
- **不能自处理** → `ActivityRelaunchItem`：**保留状态打包（onSaveInstanceState 已提前回调）→ 销毁（DestroyActivityItem）→ 全新 LaunchActivityItem 重走出生仪式 → 恢复 savedInstanceState**。

逐段精读：为什么框架"宁可重建"也不默认增量适配？——任意 View 树都可能持有与配置耦合的资源（尺寸限定符布局、横竖屏两套 layout），增量适配的正确性成本高于重建的性能成本。

**Android 在"通用正确"与"个案最优"之间永远选前者，再把优化口子（configChanges）留给开发者**——这是读框架决策的一把万能钥匙。

# 第三章 Service：最安静的组件




图 3 四大组件调度对比：入口/宿主/下发/超时四象限对照

## 3.1 启动链与"先孵化后补发"

`startService` → `ActiveServices.startServiceLocked`（`ActiveServices.java:675`）→ `bringUpServiceLocked`（`:1179`）：查 ServiceRecord → 进程不在？**先走 ProcessList 孵化**（复用书 2 的 zygote 流水线），把 ServiceRecord 挂到 `mPendingServices`；`attachApplicationLocked` 时补发 `CreateServiceItem` + `StartServiceItem`。这个"欠条-兑现"模式与 Activity 的 pending 事务、Broadcast 的 `mPendingBroadcast`、Provider 的 `mLaunchingProviders` 四处同构——**AMS 对"目标还不存在"的唯一答案就是记账后等孵化回调**。

## 3.2 bind 的三本账

`bindServiceLocked`（`:2755`）维护：ConnectionRecord（谁绑的）、ServiceRecord（服务本体）、AppBindRecord（进程间绑定桥）。回调 `onBind` 返回的 IBinder 经 `publishServiceLocked` 回传给每个连接方——**onBind 只调一次，onRebind 看意图**。死亡时 `DieConnection` 清账（书3 死亡通知的客户之一）。

## 3.3 前台服务与超时

前台服务 = 普通服务 + 类型化通知（foregroundServiceType）+ 优先级背书（PROCESS_STATE_BOUND_FOREGROUND_SERVICE）。超时线：前台 20s / 后台 200s（SERVICE_TIMEOUT/SERVICE_BACKGROUND_TIMEOUT），超时走 serviceTimeout → AnrHelper。**Android 13 的 FGS 通知延迟语义**（用户离开 10s 后才显示部分类型通知）在本树的 ActiveServices 与 SystemUI 通知过滤中实现——读码入口 fgsNotificationDeferred。

## 3.4 startServiceLocked 逐段注释（真实骨架）

java frameworks/base/services/core/.../am/ActiveServices.java:675-1179（骨架+讲解注释）

```
ComponentName startServiceLocked(IApplicationThread caller, Intent service, ...) {
// 讲解注释①：检索 ServiceRecord（首次则创建：组件名→服务账本）。
ServiceLookupResult res = retrieveServiceLocked(...);
...
// 讲解注释②：启动次数限流（后台启动 30s/30 次的熔断，防恶意自启）。
if (forcedStandby || r.startRequested) { ... }
...
// 讲解注释③：进入 bringUpServiceLocked（:1179）——"欠条-兑现"的起点。
ComponentName cmp = startServiceInnerLocked(r.defaultApp, service, ...);
}
private String bringUpServiceLocked(ServiceRecord r, ...) {
// 讲解注释④：进程在 → 直接 realStartServiceLocked：
// scheduleCreateService（onCreate）→ onStartCommand（onStartItem）
// 讲解注释⑤：进程不在 → startProcessLocked 孵化，
// ServiceRecord 挂 mPendingServices，等 attachApplication 兑现。
...
}
```

**设计洞察 · 逐段精读：三处熔断**

②的次数限流、前台服务类型强制（13 的 foregroundServiceType 必须声明）、后台启动限制（App 处于缓存态时 startService 直接抛 IllegalStateException——要求改用 startForegroundService+5s 内 startForeground）。三个限制分别对付：恶意自启、通知失明、后台耗电。**每一个"不能"，背后都有一场攻防史。**

## 3.5 onStartCommand 返回值：服务的"复活意愿声明"

java 应用侧（讲解注释）

```
public int onStartCommand(Intent intent, int flags, int startId) {
// START_NOT_STICKY：被杀后不重启（除非再有 startService）——一次性任务选它
// START_STICKY：被杀后重启，但重投的 intent 为 null（常驻服务选它）
// START_REDELIVER_INTENT：被杀后重启并重投最后的 intent（关键任务选它）
return START_STICKY;
}
```

兑现机制在 ActiveServices 的 serviceTimeout/进程死亡清理路径：ServiceRecord 记录 restartDelay（指数退避，防止杀-启死循环——SERVICE_RESTART_DURATION/SERVICE_RESET_RUN_DURATION 常量族）。**AMS 不是简单地"尊重你的声明"，而是给复活加了退避与配额**——又是一处"用户可

见语义 × 系统自保"的平衡设计。

# 第四章 Broadcast：系统的脉搏

## 4.1 双队列流水线

`broadcastIntent` 入口把接收者分流：**动态接收者进并行队列**（立即逐个分发，互不等待）；**有序接收者（含 manifest 接收者）进有序队列**（前一个 `finishReceiver` 才轮到下一个）。驱动循环：`scheduleBroadcastsLocked`（`BroadcastQueue.java:468`）→ `processNextBroadcast`（`:1137`）/ `processNextBroadcastLocked`（`:1149`）。

## 4.2 逐段精读 processNextBroadcastLocked 的四个段落

- **第一段（并行队列出货）**：while 循环把 mParallelBroadcasts 逐条 `deliverToRegisteredReceiverLocked`——每个接收者一个 Binder 调用（`scheduleReceiver`），不等回执。
- **第二段（有序队列推进）**：取队首 BroadcastRecord 的当前 receiver：进程活着→`processCurBroadcastLocked` 下发并**启动超时计时**；进程不在→孵化并挪入 `mPendingBroadcast`（记账等兑现，与 Service 同款）。
- **第三段（超时判决）**：`BROADCAST_TIMEOUT_MSG`（`:203`）到期→`broadcastTimeoutLocked`：最后手段是 ANR（前台 10s/后台 60s）。
- **第四段（收尾）**：所有 receiver 走完→finish receivers、发下一个、必要时触发后台 restriction（动态广播静默丢弃）。

> **设计洞察 · 面试题眼：为什么有序广播会引发连锁 ANR**
> 一个接收者主线程卡住→finishReceiver 迟到→超时计时到点→整条广播链 ANR。这也是 `goAsync()` 的价值：把耗时挪到自己的工作线程，但**必须在 10s 内调 finish**——它延长的是"处理窗口"不是"超时线"。

## 4.3 广播下发与超时（真实骨架逐段注释）

java am/BroadcastQueue.java（骨架+讲解注释）

```
// ── processNextBroadcastLocked 有序段落（:1149 起）──
// 讲解注释①：当前 receiver 的进程不在？孵化 + 挂 mPendingBroadcast，
// 与 Service 的 pending 同构（四组件第二处"欠条"）。
// 讲解注释②：在 → processCurBroadcastLocked：
// app.thread.scheduleReceiver(... ReceiverData ...)（一次 Binder 下发）
// + setBroadcastTimeoutLocked(lockStartCommittedElapsed)（挂超时闹钟）
// ── broadcastTimeoutLocked（超时判决）──
// 讲解注释③：前台队列 10s / 后台 60s（mTimeOutPeriod）；
// 广播期间收到 finishReceiver 会 cancelTimeout——所以 ANR 只属于"真卡住"。
// 讲解注释④：判决 = annotateAppNotResponding（走 AnrHelper 队列），
// 并跳过该 receiver 继续下一位（广播链不断，烂的只是那一环）。
```

逐段精读：④的"跳过继续"常被忽略——**ANR 杀的是进程（用户可见"应用无响应"弹窗），但广播链本身会继续推进**，系统不会因一个僵死接收者停摆。这是"单点故障不扩散"的调度哲学。

# 第五章 ContentProvider：数据的跨进程门户

`getContentProviderImpl`（`ContentProviderHelper.java:158`）是四组件里唯一**同步阻塞**的入口：发布者进程不在→孵化→**调用方线程持锁等待**（`mLaunchingProviders` 记账），`publishContentProviders` 唤醒。引用计数（`incProviderCount:261`）决定宿主进程的缓存优先级——纯 provider 进程杀得比空 Activity 进程晚的机制根源。

App 侧 `installProvider`（ActivityThread）反射建 Provider 对象并挂 `IContentProvider` 代理。**数据面不走 Binder 大包**：query 返回 Cursor（BulkCursorNative + 窗口共享内存 ashmem/memfd），大文件走 `openAssetFile`（fd 传递，书3 §6）。控制面小、数据面共享内存——这是 Provider 高吞吐的秘密。

## 5.1 getContentProviderImpl 逐段注释（同步等待的正确姿势）

java am/ContentProviderHelper.java:158-268（骨架+讲解注释）

```
private ContentProviderHolder getContentProviderImpl(IApplicationThread caller, ...) {
synchronized (mService) { // 讲解注释①：全程持 AMS 大锁——Provider 语义要求强一致
// 讲解注释②：按 authority 查 providerMap；无 → 解析 authority 找宿主包。
// 讲解注释③：宿主进程在 → 直接 installProviderLocked，把 holder 交给调用方。
// 讲解注释④：不在 → checkTime 开始计时；startProcessLocked 孵化宿主；
// mLaunchingProviders 记账；调用方 Object.wait() 让出（持锁等待），
// publishContentProviders（宿主 attach 时回调）notifyAll 唤醒。
// 讲解注释⑤：引用计数 incProviderCount（:261）——调用方进程因此
// 在 computeOomAdj 里获得 BOUND_PROVIDER 加权（宿主不容易被杀）。
}
}
```

逐段精读：④的"持锁 wait"与②的"authority 先注册"合起来构成 Provider 的**发布-订阅时序**：宿主 APK 的 provider 在 manifest 声明后由 PMS 索引（`<provider android:authorities=...>`），AMS 按 authority 寻址。超时兜底 CONTENT_PROVIDER_TIMEOUT 10s——挂起不会无限等。

App 侧消费：`installProvider` 反射建对象后，`query` 走 `BulkCursor` 协议（CursorWindow 共享内存分页搬运），大文件 `openFile` 直接 fd 过桥（书3 §6）。**控制面 Binder、数据面共享内存**——记住这句就懂了 Provider 的性能观。

## 5.2 publishContentProviders 与 URI 授权

宿主进程 attach 时（`ActivityThread.handleBindApplication` 后）调 `publishContentProviders`：把本进程全部 Provider 一次性登记进 providerMap 并唤醒等待者。**发布是一揽子的**——所以一个 App 的多个 provider 要么都可见要么都没有，不存在半发布态。

URI 级授权（`UriGrantsManagerService`，Bootstrap 批启动）提供比 provider 粒度更细的控制：`Intent.FLAG_GRANT_READ_URI_PERMISSION` 临时授权给目标包，过期自动回收（activity destroy 时清账）。**读码提示**：图库分享一张照片走的是"provider 可见性 + URI 临时票据"双闸——不是把整个图库 provider 开放给对方。

# 第六章 进程优先级：computeOomAdj 记账机器

## 6.1 computeOomAdj 因子全表（按核算顺序）

| # | 因子 | 命中后状态（例） |
|---|---|---|
| 1 | instrumentation/备份/调度器自用 | NONSTOP 高位 |
| 2 | 顶层 Activity（TOP_APP） | PROCESS_STATE_TOP |
| 3 | 可见但非顶层（被透明/PiP 覆盖） | VISIBLE |
| 4 | 可感知绑定（BIND_VISIBLE 等 flag） | BOUND_FOREGROUND_SERVICE |
| 5 | 前台服务持有 | FOREGROUND_SERVICE |
| 6 | 正在收重要广播 | IMPORTANT_FOREGROUND |
| 7 | 正在执行后台服务/普通绑定 | SERVICE / BOUND |
| 8 | 上一应用（PREVIOUS_APP） | LAST_ACTIVITY 档 |
| 9 | 缓存 provider 引用/权重提升 | BACKUP/HEAVY_WEIGHT |
| 10 | 都无命中 → 按LRU入缓存池 | CACHED_ACTIVITY(_MAX~_MIN) |

三句读表心法：**越靠前权重越大**（短路：命中即定档）；**缓存池有档位与容量**（32/32 档 min/max，超额最旧的杀）；**state 与 adj 双刻度**（state 给框架消费，adj 给 lmkd）。

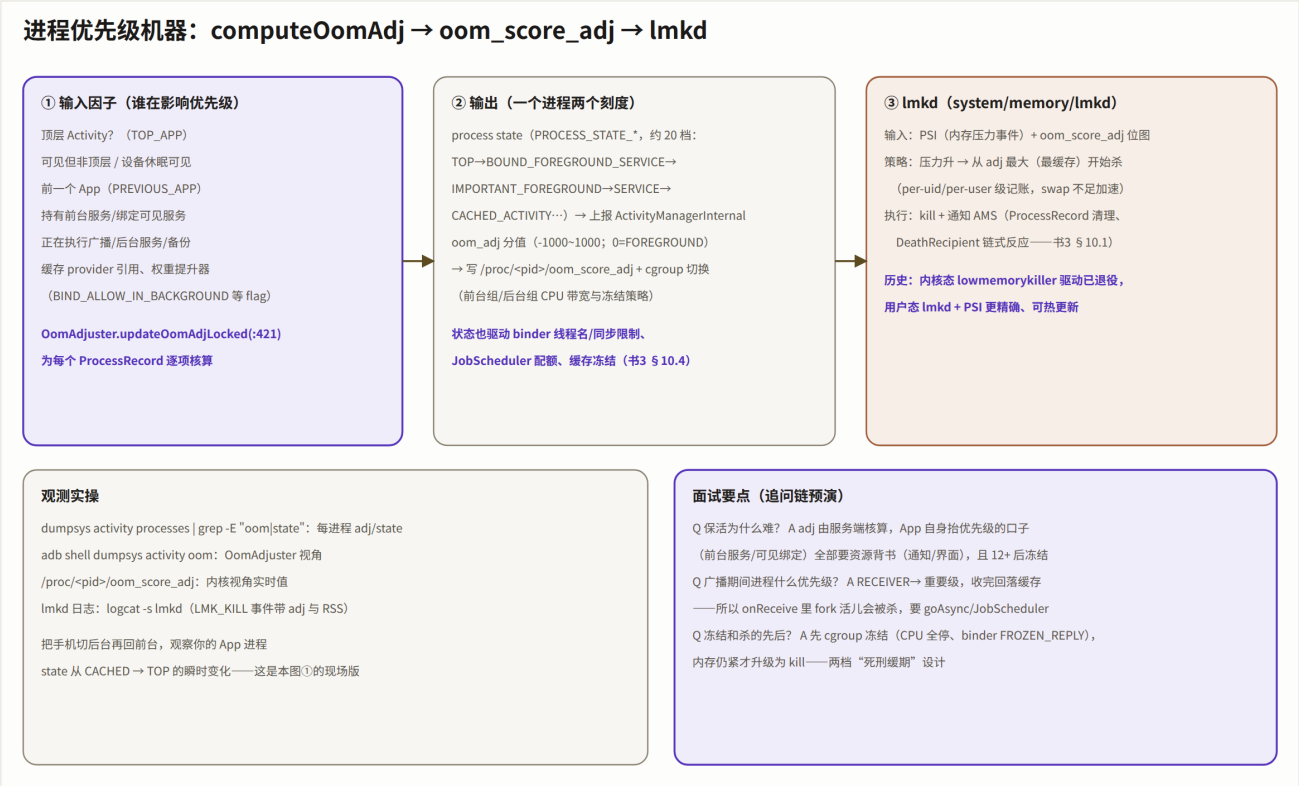



图 4 进程优先级机器：输入因子→process state/oom_adj→cgroup/lmkd，含观测与面试追问链

`OomAdjuster.updateOomAdjLocked`（`OomAdjuster.java:421`）对每个 ProcessRecord 跑 `computeOomAdjLSP`：二十来个因子（顶层/可见/前台服务/正在收广播/缓存 provider/上一应用…）核算出 `PROCESS_STATE_*` 与 `oom_score_adj`，然后三件事：写 `/proc/<pid>/oom_score_adj`、切 cgroup（前台/后台组，决定 CPU 带宽与冻结资格）、把 state 上报给各记账方（JobScheduler 配额、binder 同步限制、缓存冻结书3 § 10.4）。

**lmkd**（`system/memory/lmkd`）拿着 adj 位图与 PSI 压力事件决定杀谁：从最缓存档开始，先冻结后杀。读码入口 `AnrHelper.appNotResponding`（`:71`）则是 ANR 的统一出口——队列化执行 dumpStackTraces，避免在 binder 线程里做重活（这个细节本身就是"系统线程纪律"的活教材）。

## 6.2 cgroup 与冻结：adj 之外的两只手

adj 定生死，**cgroup 定待遇，冻结定作息**：

- **CPU 组**：前台组（`/dev/cpuctl/foreground`）与后台组（`background`，`cpu.shares` 约 2:10-20:1024 差距）；Service 客户端绑定会把宿主临时拉进前台组（`setProcessGroup` 调用族）。
- **冻结（12+）**：缓存态进程进 freezer cgroup——CPU 全停、binder 同步调用速败 `BR_FROZEN_REPLY`（书3 § 10.4）、广播直接绕行（缓存接收者广播默认 deferred）。**冻结是"杀"前的缓冲带**：内存压力小就解冻复用（省一次冷启动），压力大就顺势 kill。

lmkd 通信面：AMS 侧 `ProcessList` 经 socket 把 adj 变更推给 lmkd（`LMK_PROCPRIO` 命令），压力事件回流（`onPressureChanged`），击杀经 `LMK_KILL` 进 logcat——`logcat -s lmkd` 看到的每一行都是本节机制的现场直播。

**源码锚点 · 六册联动锚点**

<font face="MonoFT" size="9">ProcessList.java:1899</font> startProcessLocked →（书2 Zygote 流水线）<br/><font face="MonoFT" size="9">OomAdjuster.java:421</font> updateOomAdjLocked → cgroup/lmkd →（书3 §10.4 冻结）<br/><font face="MonoFT" size="9">AnrHelper.java:71</font> appNotResponding ←（书5 input-pipeline 的 5s 计时器）<br/><font face="MonoFT" size="9">ContentProviderHelper.java:158</font> ←（书6 PMS 的包宇宙支撑解析）

# 第七章 深水区：ART 内存执行系统与 lmkd

## 7.1 ART GC：并发复制收集器（CC）

主收集器是**并发复制**（`art/runtime/gc/collector/concurrent_copying.cc`，堆按 RegionSpace 的 region 组织）：标记与对象搬移大部分与应用线程并发，STW 只剩根初始标记与最终修正两个小阶段。支撑三件套：**读屏障**（引用读取顺带看 forwarding 指针、按需搬移对象——搬移不停止世界）、**卡表/记忆集**（跨代引用脏页索引，免全堆扫）、**LOS 大对象空间**（独立回收防碎片拷贝）。观测投影即 `dumpsys meminfo` 的各 space 行。

**设计洞察 · 面试一句话**

Dalvik 是标记-清除 STW；ART CC 把停顿压到毫秒级，代价是每条引用读都带屏障——**用 CPU 换延迟**，与 Binder 用一次拷贝换安全是同一种设计味道。

## 7.2 MTERP：为什么解释器用汇编写

`art/runtime/interpreter/mterp/`（arm64ng）：每条字节码一个汇编 handler，取指-分发-执行全内联、虚拟寄存器钉物理寄存器——不走 C 调用栈，吞吐近翻倍。它不是 JIT（仍逐条解释），是把"解释的地板"垫高；JIT（`art/runtime/jit/`）在此之上按轮廓计数挑热点进 code cache，profile（书1 §8.1 的反哺环）供 AOT 精编。**解释器-快冷、JIT-快糙、AOT-慢精**三级火箭各司其职。

## 7.3 lmkd：压力驱动的击杀守护

`system/memory/lmkd/lmkd.cpp:3735` main：订阅内核 **PSI** 压力事件（libpsi）+ 接收 AMS 的 adj 变更（LMK_PROCPRIO 命令族）；压力升级时按 adj 位图从最缓存档选受害者，配合 per-uid 约束与 swap 水位。一句话：**AMS 出价（adj）、内核报时（PSI）、lmkd 落槌**。`logcat -s lmkd` 的 Kill 行即成交记录。

## 7.4 冻结实现与 cgroup 拓扑

**CachedAppOptimizer**（am/CachedAppOptimizer.java）：缓存进程整体迁入 freezer cgroup（v2），调度组件前 AMS 解冻（宽限窗口）；冻结态的 binder 行为即书3 §10.4。**cgroup 单一事实源**：`system/core/libprocessgroup/profiles/cgroups.json`（层级挂载）+

task_profiles.json（命名档：ProcessCapacityHigh 等——书2 zygote rc 里的 task_profiles 行引用的就是它）。**zram**：rootdir/init.rc:1074-1076 暴露 idle/writeback，system_server 管理回写——压缩换内存的标配。

> **源码锚点 · 本章锚点**
> <font face="MonoFT" size="9">art/runtime/gc/collector/concurrent_copying.cc</font> CC 主体 / <font face="MonoFT" size="9">interpreter/mterp/arm64ng/</font> 汇编解释器 / <font face="MonoFT" size="9">jit/</font><br/><font face="MonoFT" size="9">system/memory/lmkd/lmkd.cpp:3735</font> main + libpsi<br/><font face="MonoFT" size="9">am/CachedAppOptimizer.java</font> 冻结状态机<br/><font face="MonoFT" size="9">libprocessgroup/profiles/{cgroups,task_profiles}.json</font> cgroup 双源

# 第八章 线程发动机：Handler/Looper/MessageQueue 全解

本丛书在书2（Looper.prepareMainLooper）与书5（同步屏障）两处直接使用 Handler 体系却未拆解它。它既是 App 开发者的老朋友，也是 system_server 全部服务的心脏——值得一次讲透。

## 8.1 三件套的分工与源码骨架

java frameworks/base/core/java/android/os（骨架+讲解注释）

```
// Handler：发件与收件一体
public boolean sendMessageAtTime(Message msg, long uptimeMillis) { // :719
// 讲解注释①：消息按"内核单调时钟的执行时刻"入队（不是 delay！），
// MessageQueue 按 when 排序——定时消息与普通消息同一条队。
...
}
public void dispatchMessage(Message msg) { // :97
// 讲解注释②：回调优先级：msg.callback(Runnable) > mCallback > handleMessage()。
// post(Runnable) 与 sendMessage 的差异就消失在这两级回退里。
}
// Looper.loop()（:260）：for(;;) { queue.next(); msg.target.dispatchMessage(msg); }
// —— 主线程"死循环不卡 UI"的真相：循环里的 next() 会睡眠（下节）。
```

## 8.2 关键一层：Java 队列的底部是 native epoll

java MessageQueue.java（节选+讲解注释）

```
private native void nativePollOnce(long ptr, int timeoutMillis); // :72
private native static void nativeWake(long ptr); // :73
...
nativePollOnce(ptr, nextPollTimeoutMillis); // :335 next() 内：无消息→睡到下个 when
...
nativeWake(mPtr); // :443 enqueueMessage 唤醒（若有更早者/睡眠中）
```

**往下再一层**（core/jni/android_os_MessageQueue.cpp:41 NativeMessageQueue → system/core/libutils/Looper.cpp:181 pollOnce → :245 epoll_wait）：MessageQueue 持有

native Looper，epoll 监听三样——wake 管道（nativeWake 写它）、空闲处理器 fd、以及**别的线程塞进来的自定义 fd**（这正是 InputChannel 能被 ViewRootImpl 收到事件的底座：书5 §5.2 的输入回 调就是"消息队列顺带 epoll 了 socket"）。所以准确地说：**Handler 是"用 epoll 武装的优先级队列"**；书5 同步屏障插队、书2 system_server 主循环、Choreographer 的 vsync 消息（异步消息）都是 这同一台发动机的不同挂挡。

## 8.3 系统侧的三种用法对照

| 用法 | 现场 | 要点 |
| --- | --- | --- |
| system_server 主线程 | 书2 :860 prepareMainLooper | 服务构造在主线程、binder 事务转主线程执行 |
| HandlerThread | AMS/ATMS 各线程域 | 后台线程自带 Looper（binder 线程≠Handler 线程，桥靠 post） |
| binder→handler 中转 | ApplicationThread:1847 | binder 线程收事务→主线程 handleMessage（ClientTransaction 的 App 侧落点） |

**源码锚点 · 本节锚点**

<font face="MonoFT" size="9">Handler.java:97/719</font> / <font face="MonoFT" size="9">Looper.java:260</font> / <font face="MonoFT" size="9">MessageQueue.java:72/73/335/443</font><br/><font face="MonoFT" size="9">android_os_MessageQueue.cpp:41/107</font> / <font face="MonoFT" size="9">libutils/Looper.cpp:181/245</font><br/>衔接：书5 §3.1 同步屏障 = 本节队列的"异步消息优先"开关

# 第九章 设计决策八问

## 9.5 三个高频补全：PendingIntent、onActivityResult、动态注册桥

**PendingIntent**（`PendingIntent.java:432` getActivity 族）：把"意图+创建者身份+权限"打包成可跨进程转交的凭据——AMS 侧 `getIntentSender` 落账为 **PendingIntentRecord**（`am/PendingIntentRecord.java:54`，`IIntentSender.Stub`）。通知/闹钟/Widget 持有它，触发时由持有者进程调用 send，**以创建者（通常是你 App）的身份与权限执行**——"代扣授权"的系统级实现。安全要点：IntentSender 可取消（cancel）、可变性（FLAG_IMMUTABLE，13 默认强制）、越权面（持凭据者即得你的身份）。

**onActivityResult 回传链**：startActivityForResult 的 requestCode 随 LaunchActivityItem 到达目标（ActivityRecord 记账）；目标 `setResult+finish` → AMS 生成 **ActivityResultItem**（servertransaction/ 目录，与 LaunchActivityItem 同族）在下次 resume 发起方时随事务下发 → `ActivityThread.handleSendResult` → onActivityResult。所以"结果回来的时机=你重新可见的时机"——背后的执行器还是书4 §2 的同一台事务机器。

**动态注册桥（ReceiverDispatcher）**：`registerReceiver` 时 LoadedApk（`core/java/android/app/LoadedApk.java:107`）为每个 Context+Receiver 组合建 **ReceiverDispatcher**（内部类）：持 IIntentReceiver binder stub + 一个 Handler（默认主线程）。BroadcastQueue 分发到它 → binder → dispatcher 的 Handler post → 主线程 onReceive。这解释了两个高频事实：**动态接收者永远在主线程回调**（onReceive 里别干重活）与**注册与分发靠 dispatcher 记账防泄漏**（unregister 才拆 binder）。Service 侧同构的 ServiceDispatcher 管绑定回调。

- **为什么四大组件都要"记账-孵化-补发"？** 目标进程可能不存在；事务性调度把"不存在"变成可兑现的欠条，而不是失败。
- **为什么 Activity 调度拆到 ATMS/wm？** 2019 分家：窗口/焦点语义与进程调度解耦（书5 双视图架构），AMS 只管进程，ATMS 管画面。
- **为什么事务化生命周期？** 抗乱序、抗闪烁、批量化——一次 Binder 说清"发生什么+终到哪"。
- **为什么 Provider 同步等待而其他组件异步？** 调用方语义上就要数据（query 返回值），无法欠条化；代价是 10s 超时兜底。
- **为什么广播分双队列？** 动态接收者在乎吞吐（并行），有序接收者在乎次序（串行）——两种业务诉求两种队列。
- **为什么 oom_adj 由服务端核算？** 若 App 自报，保活将不可治理；核算所需的可见性/绑定关系只有 AMS 视角完整。
- **为什么 ANR 统一走 AnrHelper 队列？** ANR 处理本身重（dump 全线程栈），在触发线程同步做会放大卡顿——系统也遵守"主线程不做重活"。
- **为什么 onStartCommand 要三种返回值？** 复活意愿是业务语义，只有开发者知道；框架负责的是给它加上退避与配额的护栏。
- **为什么旋转默认重建？** 通用正确性优先于个案性能，优化口子交给 configChanges 声明。
- **为什么缓存进程冻结而非直接杀？** 冷启动是最贵的用户可感知操作；冻结是"可逆的杀"，内存压力决定是否兑现。
- **为什么 bindService 回调要走 ServiceConnection 而非返回值？** 绑定是异步语义（进程可能要孵化），回调给了框架调度自由。
- **为什么前台服务要通知？** 用户必须能看见"谁在耗电运行"；通知是优先级背书的对价——13 的延迟显示只是弱化打扰，不是取消透明度。

# 第十章 实战：读一次真实启动的 trace

把 `am start -W` 与 systrace 拼起来读（示例输出，字段名真实）：

text am start -W 输出解读

```
Status: ok
LaunchState: COLD ← 冷启动（另有 WARM/HOT）
Activity: com.example/.MainActivity
```

```
TotalTime: 1420 ← 应用可见总耗时（首帧口径）
WaitTime: 1533 ← 调用方视角（am 进程等待总时长）
Complete: true
```

对应 systrace 轨道逐段认领：`am_proc_start`（孵化，§1 图④）→ `ActivityManager: activityStart`（⑥⑦）→ `inflate/Choreographer#doFrame *first*`（⑧⑨，衔接第五册）→ `am_proc_died`（若被杀）。**把 TotalTime 按轨道切成"孵化/事务/首帧"三段**，你就拥有了对任何启动缓慢的定位能力——这也是性能优化团队的第一课。

一个练习闭环：在测试机反复 `am start -W` 同一 App，记录五组 TotalTime；然后 `pm clear` 再测（首启动 vs 热数据）；最后对照本册十步图解释每组差异来自哪几步。

# 附录 A 观测实操

| 目标 | 命令 | 对应机制 |
| --- | --- | --- |
| 启动全链 | am start -W -n pkg/Activity | WaitTime/TotalTime 分段 |
| 组件全景 | dumpsys activity activities / services / broadcasts / providers | 四组件各自的 dump |
| 进程优先级 | dumpsys activity processes + /proc/<pid>/oom_score_adj | computeOomAdj 现场 |
| ANR 现场 | /data/anr/ + dropbox | AnrHelper.dumpStackTraces |
| 广播队列 | dumpsys activity broadcasts | BroadcastQueue 内景 |
| 杀进程链 | logcat -s lmkd am_kill | lmkd→AMS 清账 |

# 附录 B 自检题

- 默画冷启动十步，并标注每步所在进程。
- ClientTransaction 的 callbacks 与 lifecycleStateRequest 各承担什么？cycleToPath 补的是什么？
- 四组件的"欠条-兑现"分别叫什么名字（四个 pending 容器）？
- computeOomAdj 的输入因子列五个；前台服务把进程抬到哪一档？
- 有序广播 ANR 的传导链是什么？goAsync 改变了链条中的哪一环？

# 附录 C 崩溃/卡顿与调度的对照表

| 症状 | 调度层解释 | 排查入口 |
| --- | --- | --- |
| ANR Input（5s） | 主线程没回执输入事件（书5 §7） | traces 主线程栈 |

| 症状 | 调度层解释 | 排查入口 |
| --- | --- | --- |
| ANR Broadcast（10s/60s） | onReceive 未 finish | BroadcastQueue 超时接收者 |
| ANR Service（20s/200s） | onCreate/onStartCommand 超时 | ActiveServices 计时 |
| ANR Provider（10s） | 宿主未及时 publish | ContentProviderHelper 计时 |
| IllegalStateException: Not allowed to start service | 缓存态后台 startService 熔断 | 改 startForegroundService |
| ForegroundServiceDidNotStartInTimeException | 5s 内未 startForeground | FGS 通知链 |
| Activity 泄漏 | 静态持有 Activity 引用，但调度层早已 destroy | leaks 工具+DestroyActivityItem 路径 |
| 重复 onCreate 而非 onNewIntent | launchMode/flags 判决为 standard | §1.5 裁决顺序表 |

# 附录 D 快问快答（面试扫雷 20 条）

AMS 与 ATMS 的分界？——进程调度（am/）vs 任务与窗口（wm/），2019 拆分。 冷/暖/热启动定义？——进程新建/已存在新 Activity/仅 resume；am start -W 的 LaunchState 直接标出。 - onNewIntent 触发条件？——launchMode 判决复用实例（singleTop 栈顶等）。 - ActivityRecord 与 ActivityClientRecord？——系统侧/App 侧镜像，token 为桥。 - ClientTransaction 一次能带几个回调？——多个（callbacks 列表），终态一个。 - 为什么 Application.onCreate 可能晚于首个 Activity 的部分初始化？——makeApplicationInner 在 performLaunchActivity 内触发。 Service onBind 返回 null 会怎样？——绑定成功但收不到 binder（onServiceConnected 拿 null）。 stopService 与 unbindService 组合语义？——started/bound 两本账独立，双清才销毁。 广播防自启限流？——后台 30s/30 次（ActiveServices/AMS 双限流）。

有序广播能截获吗？——abortBroadcast 仅 ordered 链有效。 goAsync 的正确用法？——PendingResult.finish() 必须在超时线内调。 ContentProvider 的 multi-process 声明？——android:multiprocess 极少用（进程内直连，失去统一账本）。 query 大表为什么不卡 Binder？——CursorWindow 共享内存分页。 oom_adj 与 nice 的区别？——内存查杀依据 vs CPU 调度优先级（两套体系，cgroup 才是 CPU 待遇）。

缓存进程多少档？——CACHED_ACTIVITY_MIN~MAX 分组限容，LRU 逐档淘汰。 冻结中的 App 能收广播吗？——默认 deferred，解冻补投。 LMK 与 lmkd？——内核驱动退役，用户态守护 + PSI。 ANR 弹窗谁弹？——AMS 侧 showAnrUi 经权限校验拉起系统对话框进程。 am kill 与 kill -9 差别？——走 AMS 清账（组件/连接/notification 同步回收）。

学完本册下一步？——第五册（窗口与输入）承接"首帧之后"，第六册（PMS）承接"包从哪来"。